

EZ Controller (IR-CT01)

사용자매뉴얼

EZ Controller (IR-CR01)



목 차

01 개요	3
1.1. 사용 시 주의사항-----	3
1.2. 적절한 보관-----	3
02 기능 및 기본 동작	4
2.1. 제품 기능설명-----	4
2.2. 모드선택 -----	5
2.3. 외부 스위치 및 신호 입력-----	5
2.4. 파트 핀맵-----	6
03 아두이노 개발환경 구축하기	7
3.1.아두이노 IDE 설치-----	7
3.2.아두이노 IDE 기본 구성-----	10
3.3.아두이노 라이브러리 추가-----	10
3.4.예제 불러오기-----	11
3.5.프로그램 업로드 하기-----	11
04 아두이노 IDE 를 통한 액추에이터제어 예제	12
4.1. 개 요-----	12
4.2. 예제 -2 Position Control -----	12
4.3. 예제 - Toggle Position-----	14
4.4. 예제 - Manual Position-----	15
4.5. 예제 - Basic Function-----	16
4.6. 예제 - Extra IO(1)-----	17
4.7. 예제 - Extra IO(2)-----	18
4.8. 예제 - External Communication-----	19
4.9. 예제 - Mode Selection -----	20
4.10. 예제 - Stroke Limit-----	21

1 개요

Arduino 기반의 EZ Controller는 mightyZAP 리니어 액추에이터를 손쉽게 동작 / 테스트해보기 위한 컨트롤러입니다.

EZ Controller의 특징은 다음과 같습니다.

- 제품 구매 시 이미 내장되어 있는 기본 프로그램을 이용하여 보드 상의 여러 입력 장치를 통해 리니어 액추에이터를 간단히 손쉽게 동작시킬 수 있습니다.
- 당사가 제공한 다양한 예제 프로그램을 이용하거나, 아두이노 프로그램을 통해서 원하는 프로그램을 자유롭게 직접 만들어, 원하는 동작을 만들어낼 수도 있으며, 목적에 맞는 다양한 제어가 가능합니다.
- 다양한 기능의 외부 악세서리들이 연결 가능한 구조입니다.
- MightyZAP 리니어액추에이터를 제어하기 위한 별도의 회로 구성이 필요 없으며 전원 역압 방지, 과전류 방지, 정전기 방지 등 전원 보호회로가 내장되어 있어 안전하게 사용하실 수 있습니다.

1.1 사용시 주의 사항

아래 주의사항은 사용시 각별히 주의를 요하는 사항이므로, 반드시 숙지를 하여 주십시오. 아래 사항을 준수하지 못해 발생한 문제에 대해서는 보증서비스를 받을 수 없음을 알려드립니다.

1. 제품 연결 시 **무리한 힘으로 결합하지 마십시오.** 또한, 소자 또는 외부로 돌출된 핀에 무리한 힘을 가하지 마십시오. 고장의 원인이 됩니다.
2. 절연이 되어 있지 않은 금속판 위에서 전원 연결 시 단락이 될 수 있습니다. 반드시 **동봉된 플라스틱 서포트를 기판에 장착 후 사용하여 주시기 바랍니다.** (우측 그림 참조)
3. 이지 컨트롤러에 인가된 전압이 연결 액추에이터에 바로 인가 되므로, **액추에이터의 구동 전압범위를 확인하시고, 외부전원 연결 시 극성을 확인하십시오.**
예) 입력 전압 범위가 6.5V - 8.9V인 7.4V 인가 전압의 액추에이터 (D7또는 L7계열) 연결 시, 실수로 12V전원을 연결하면 액추에이터가 구동을 하지 않고 파손 위험이 있습니다.
4. 제품에 액추에이터 연결 시 **핀의 극성을 꼭 확인하고 결합하십시오.**
극성이 바뀔 경우 모터 손상의 원인이 됩니다
5. **알맞은 규격의 커넥터를 사용하십시오.** 액추에이터와 연결 시 오삽에 따른 제품 파손의 위험이 있습니다.
6. **각 단자에 용도에 맞는 입력을 넣어주십시오.** 오동작의 원인이 됩니다.
알맞은 전압레벨의 신호단자와 전원/신호 단자의 혼란에 유의 하십시오.
7. 로드 시작 및 종료 지점을 설정하기 위한 **가변저항의 노브를 과도하게 많이 돌리지 마십시오.**
8. 제품을 화기, 물기, 먼지, 기름으로부터 멀리하십시오.
9. 본 제품은 실내용으로 설계된 제품입니다. 실외에서의 사용은 최대한 자제하여 주십시오.
10. 어린이의 손이 닿지 않는 곳에 제품을 보관하십시오.



1.2 적절한 보관

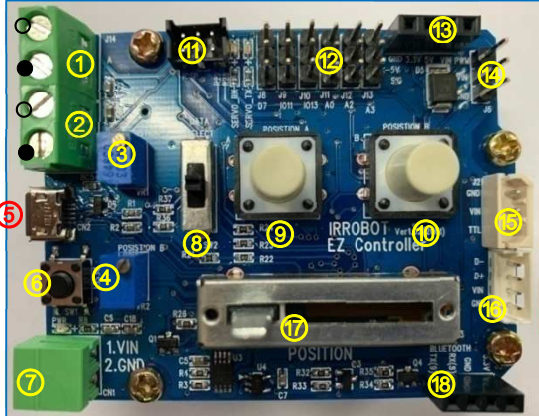
아래와 같은 극한환경에서 제품을 사용하거나 보관하지 마십시오. 오작동이나 제품의 파손을 불러올 수 있습니다.

- 섭씨 60 도 이상의 고온환경 또는 섭씨영하 20 도의 극저온환경
- 직사광선 또는 화기근처
- 고온다습하고 먼지가 많은 장소
- 진동이 심한 장소
- 정전기를 유발할 수 있는 장소

2 제품 기능 및 기본동작

2.1 제품 기능 설명

■ EZ Controller 구성 부품 및 설명



/Serial 통신

- ⑥ Reset 스위치 : 컨트롤러의 동작에 이상이 있을 시, 동작을 초기화
- ⑦ 12V 메인 전원 입력 단자
- ⑧ Mode 변환 스위치
- ⑨ A Position 푸쉬 버튼 스위치
- ⑩ B Position 푸쉬 버튼 스위치
- ⑪ MCU F/W 다운로드 커넥터(ICSP) : **사용자 조작 금지**
- ⑫ 아두이노 I/O 핀(Digital:3 , Analog:3 / GND, 12V, SIG)
- ⑬ 내부 전원 단자(GND / 3.3V / 5V / 12V) : 내부에서 사용하는 전원 출력 단자
[각 단자의 Current Limit]
- 3.3V : ~150mA / 5V : ~ 900 mA / 12V : Depending on Input Voltage Source
- ⑭ 리니어액추에이터 PWM 포트
- ⑮ 리니어 액추에이터 TTL 포트
- ⑯ 리니어 액추에이터 RS-485 포트
- ⑰ 리니어 액추에이터 위치 조작 Potentiometer
- ⑱ 사용자 외부 통신 단자 (Bluetooth : TX / RX / GND / 3.3V)

- ① A Position 외부 스위치 입력(O: +, ●:-)
- ② B Position 외부 스위치 입력(O: +, ●:-)
: ①,②번 단자는 외부 스위치 연결 또는 외부 입력 신호를 받아 동작 (⑨,⑩번 푸시버튼과 같은 기능이며 포지션 설정은 마찬가지로 ③과④를 통해 설정함)
- ③ A Position 설정 가변저항 :: 가변 저항을 조절하여 ① 또는 ⑨번 입력 시 액추에이터가 움직일 Position을 설정
- ④ B Position 설정 가변저항 :: 가변 저항을 조절하여 ② 또는 ⑩번 입력 시 액추에이터가 움직일 Position을 설정
- ⑤ PC 연결 마이크로 USB 단자 : 아두이노 스케치 다운

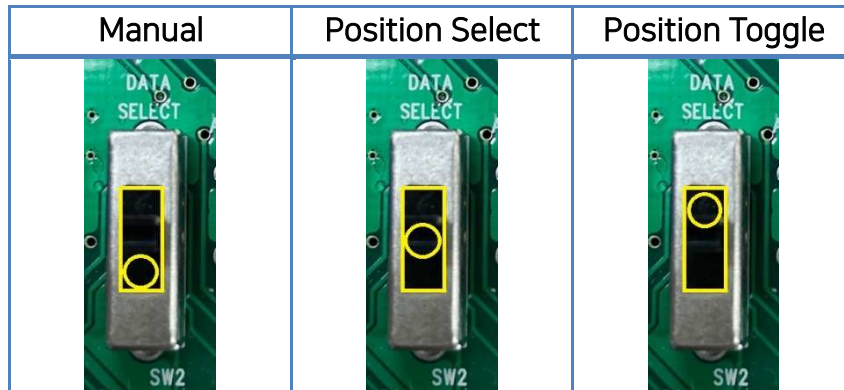
청색 가변저항 ③(A포지션), ④(B포지션)를 각각 시계·반시계 방향으로 조절하여 A와 B지점의 위치를 조정.

- 시계 방향 : - (액추에이터 수축방향)
- 반시계 방향 : + (액추에이터 확장방향)

편의상 아래와 같이 A는 최소위치, B는 최대 위치로 설정하는 것이 편하며,
-A 지점(부품③) : 시계(수축) 방향으로 돌려 최소 위치 설정
-B 지점(부품④) : 반시계(확장) 방향으로 돌려 최대 위치 설정
다만, 사용자 설정에 따라 A와 B의 최소/최대위치가 역전되어 변경될 수도 있습니다.

2.2 모드 선택

- EZ Controller에 내장되어 있는 기본동작 프로그램을 이용하여 액추에이터의 동작을 테스트 합니다.
- 기본 동작 프로그램은 Manual Mode, Position Select Mode, Position Toggle Mode로 구성되어 있습니다.
(모드스위치는 기본 내장된 프로그램에서만 영향을 미칩니다.)



- **Manual Mode** : 액추에이터를 연결하고 Mode 스위치를 하단으로 내려 Manual Mode로 설정하고 중앙부 선형 Potentiometer의 위치를 변화시켜 액추에이터를 조작할 수 있습니다.
- **Position Select Mode** : 액추에이터를 연결하고 Mode 스위치를 중간단으로 올려 Position Select Mode로 설정하고 파란색 포지션 설정 가변 저항을 조절하여 A, B Position을 각각 설정한 후, 중앙부 흰색 Push Button을 눌러 액추에이터를 A지점과 B지점으로 동작시킬 수 있습니다.
- **Position Toggle Mode** : 액추에이터를 연결하고 Mode 스위치를 상단으로 올려 Position Toggle Mode로 설정하고 파란색 포지션 설정 가변 저항을 조절하여 A, B Position을 각각 설정한 다음 중앙부 흰색 Push Button 둘중 아무 버튼을 눌러 액추에이터를 A지점과 B지점으로 반전 동작시킬 수 있습니다.

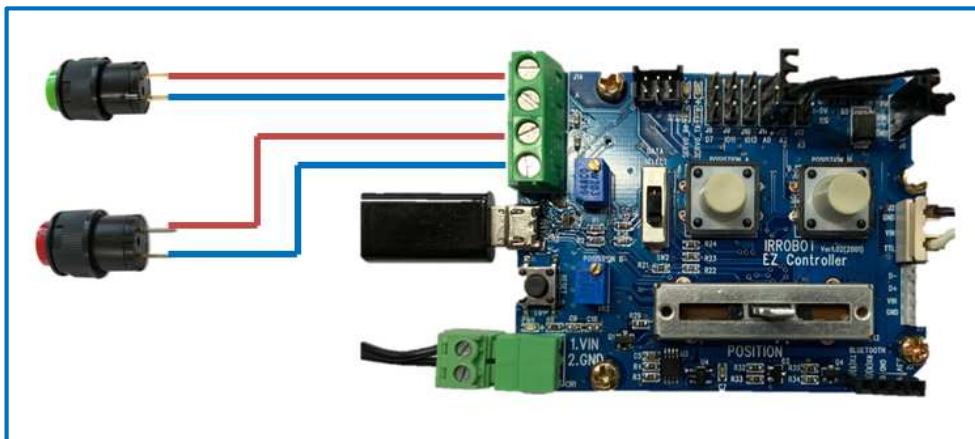
2.3 외부 스위치 및 외부신호 입력

A/B 포지션 외부 스위치 및 신호 입력을 통한 위치명령 (그림 ①,②번 단자)

스위치를 연결하거나, 또는 전압레벨 신호를 인가하여 위치이동 명령을 내립니다. 흰색의 푸쉬버튼 스위치(⑨, ⑩)와 동일한 기능을 합니다. 위치 명령만 내릴뿐, 위치 지정은 청색 가변저항(③과④)을 통해 셋팅을 합니다.

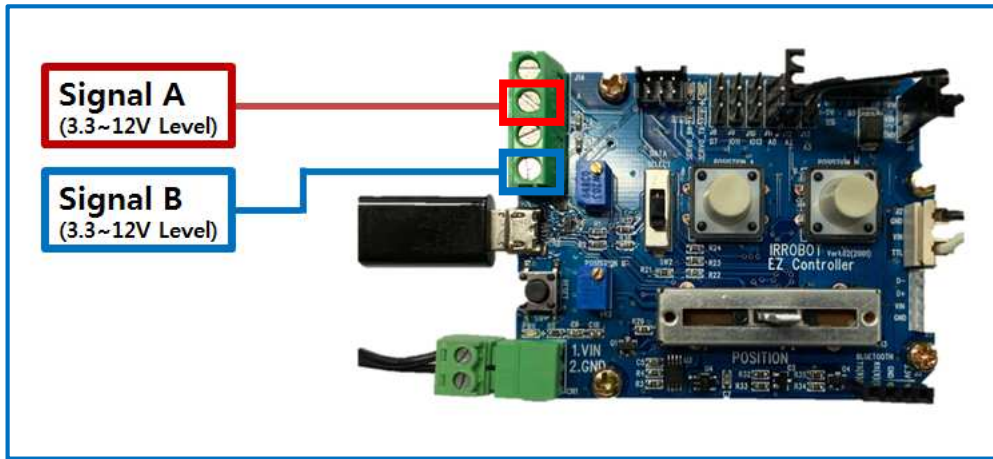
1) External Switch 연결

: 단자의 양단을 스위치를 통해 연결시켜주어 신호를 입력해줍니다. (단자 양단 단락)



2) External Signal 입력

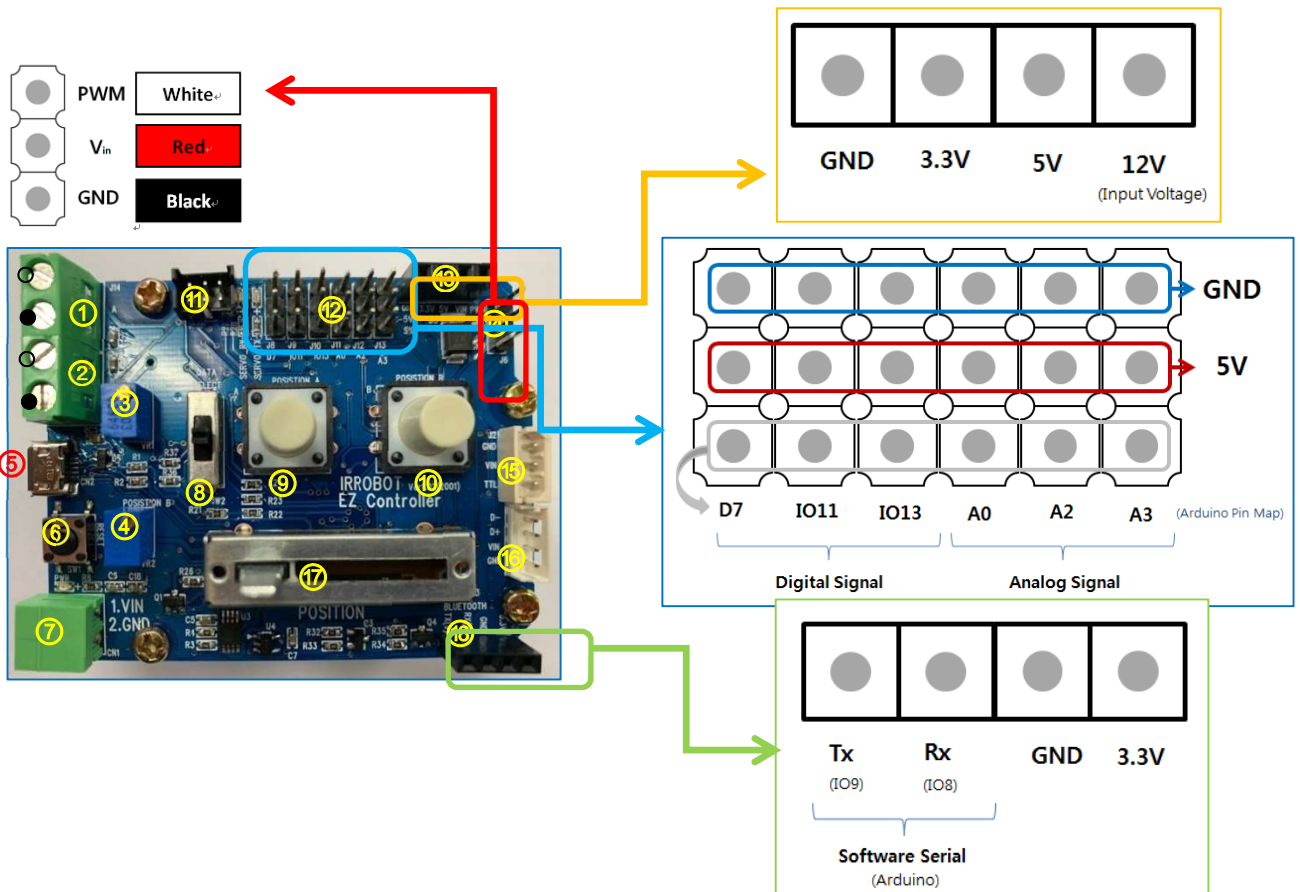
: 단자의 두번째 홀에 신호 인식을 위하여 3.3~12V 전압레벨의 신호를 인가시켜줍니다.



2.4 파트 핀맵

- Parts No : 구성부품 표기 번호
- Pin Name : Arduino(Leonardo) I/O핀

Part No	Pin Name	Purpose	Part No	Pin Name	Purpose
①,⑨	I/O10	GPIO- A Signal	⑫	A0,A2,A3	GPIO - Analog
②,⑩	D3	GPIO- B Signal	⑭	D5	GPIO - PWM
③	A4	Analog-A 가변저항	⑮,⑯	D0,D1	TTL,RS485 통신 전용
④	A5	Analog-B 가변저항	⑰	A1	Analog -Potentiometer
⑤	Serial	USB Serial, Sketch다운	⑱	I08,I09	Software Serial
⑧	D4,I012,D6	GPIO - Slide SW			
⑫	D7,I011,I013	GPIO - Digital			



3 아두이노 개발환경 구축하기

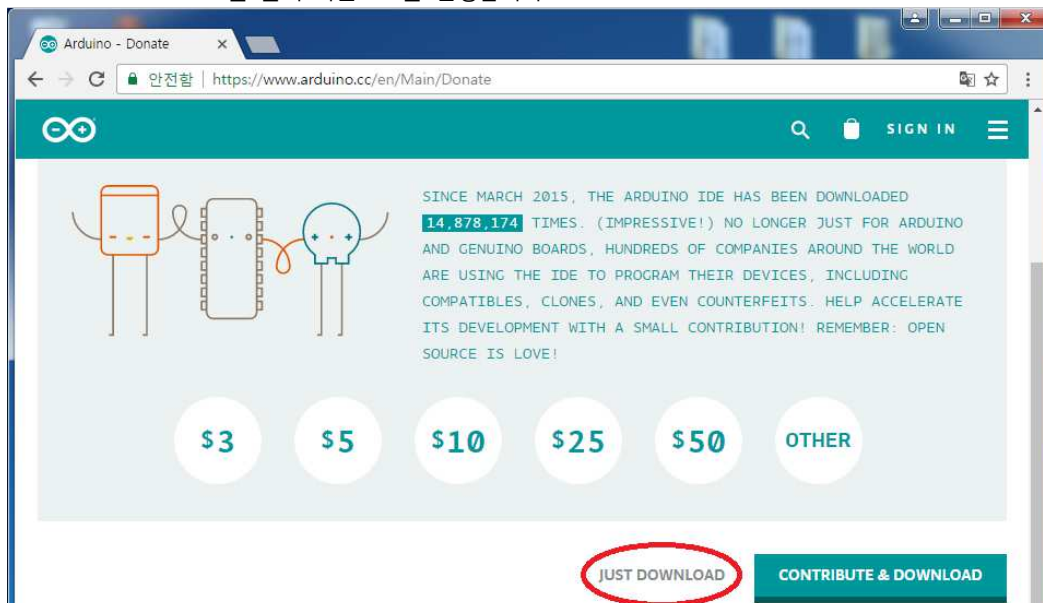
IRROBOT Ez Controller 를 통해 테스트 프로그램을 동작시키기 위해선 PC 에 아두이노 개발 환경을 구성해야 합니다.

3.1. 아두이노 IDE 설치

1. <https://www.arduino.cc/en/main/software> 에서 Window Installer를 선택



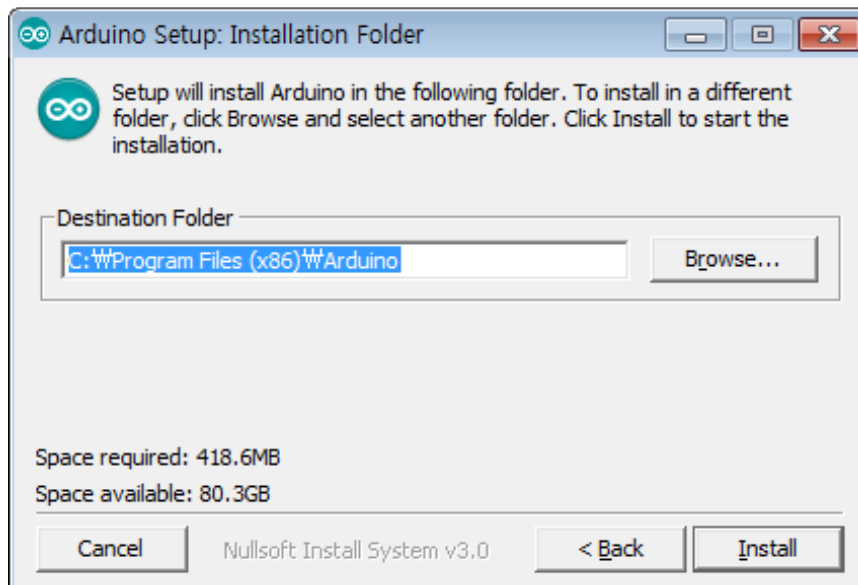
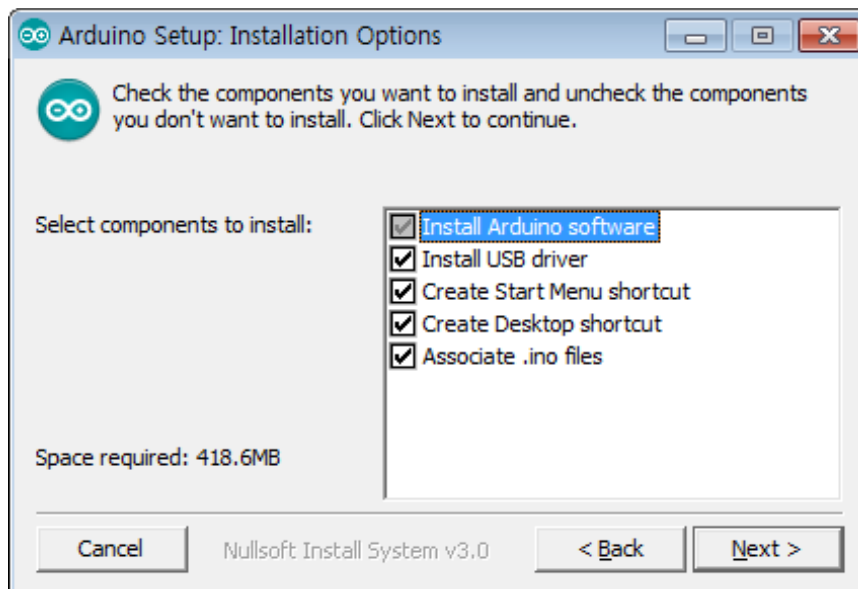
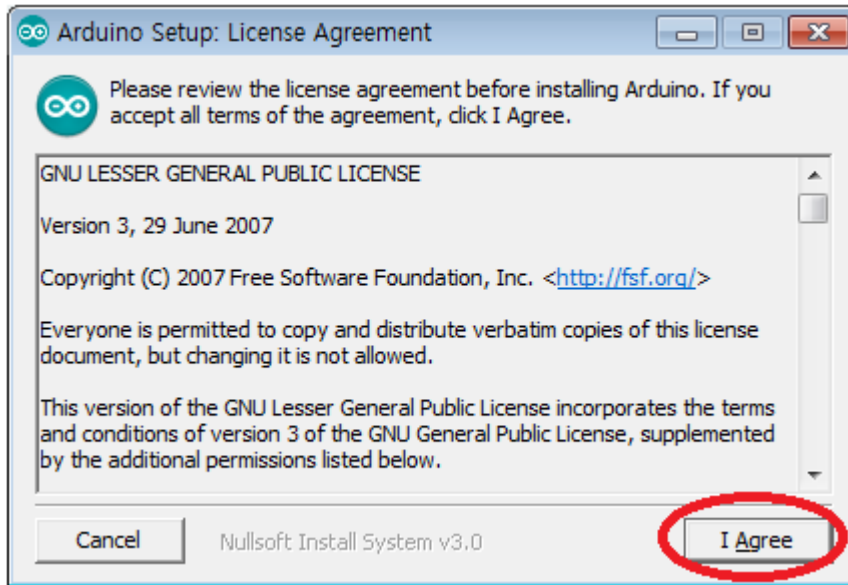
2. JUST DOWNLOAD를 눌러 다운로드를 진행합니다.

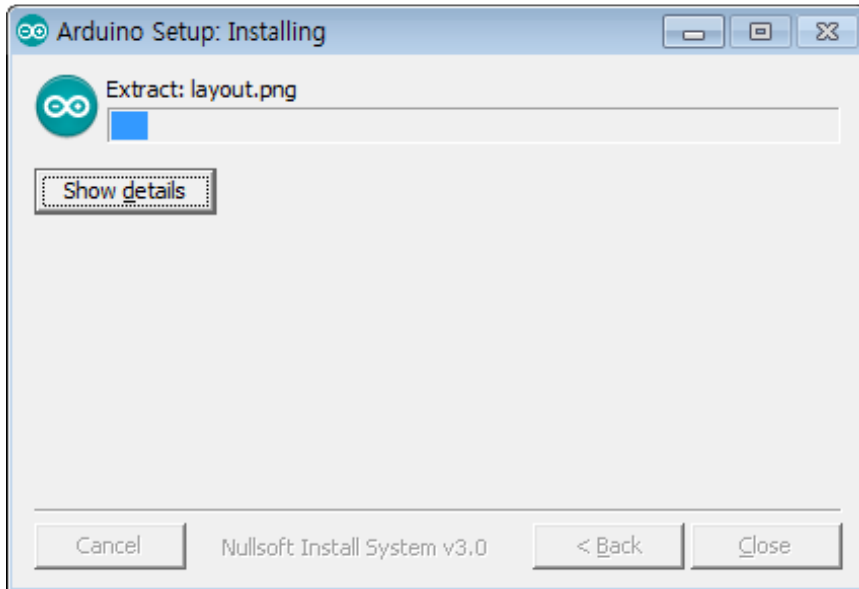


3. 다운로드가 완료 되면 arduino-xxx-windows.exe를 실행 합니다.

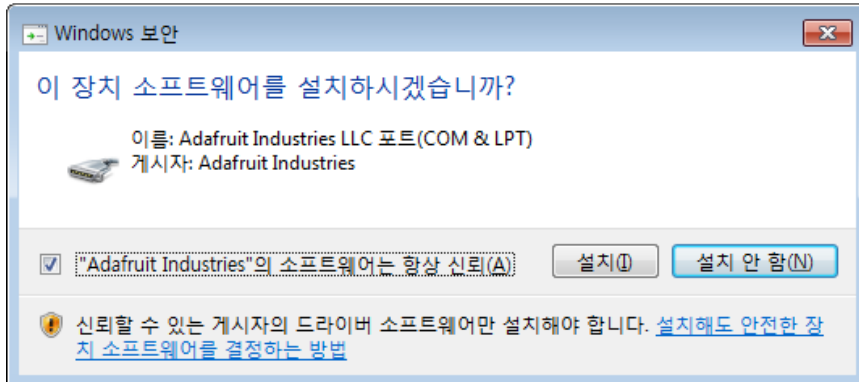


4. 다음의 과정을 따라 소프트웨어를 설치를 진행 합니다.

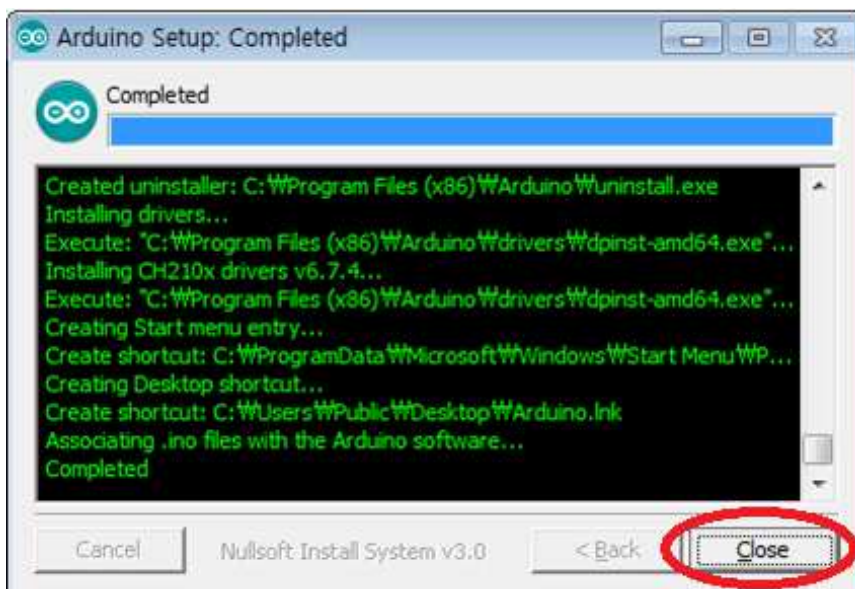




5. 설치 과정에서 나오는 추가 드라이버를 설치합니다.



6. 소프트웨어 설치가 완료되면 Close 를 누릅니다.

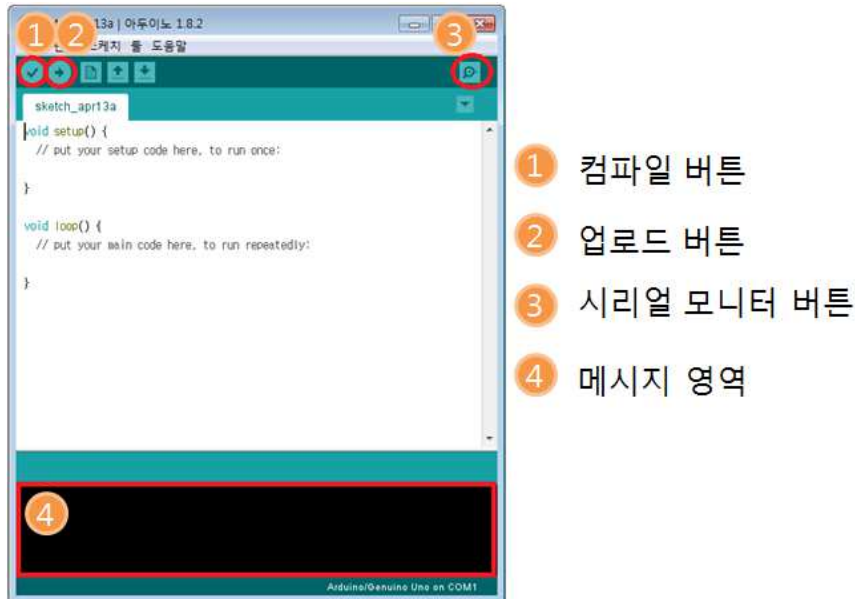


7. 바탕화면에 생성된 아두이노를 실행 합니다.



3.2. 아두이노 IDE 기본 구성

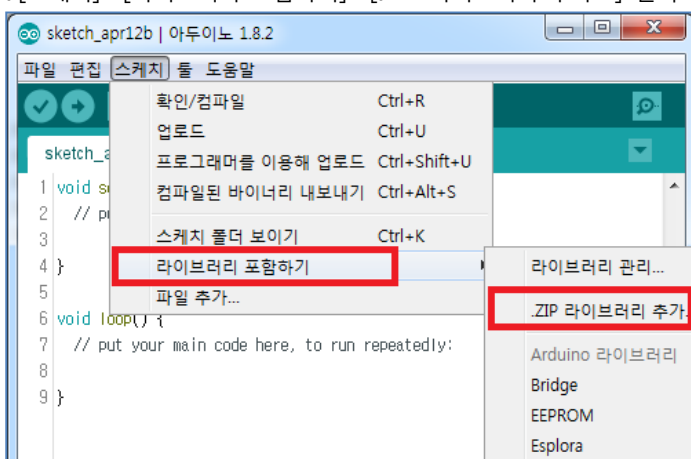
아두이노 개발환경 기본 구성은 다음과 같습니다.



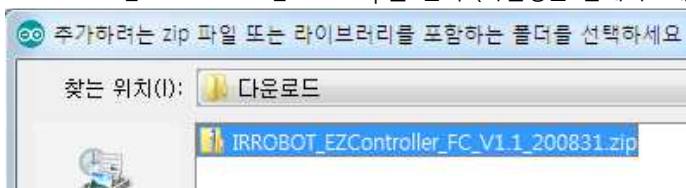
1. 컴파일 버튼 : 이 버튼을 누르면 프로그램이 컴파일 합니다.(컴파일 결과는 4번에 표시)
2. 업로드 버튼 : 컴파일과 동시에 Arduino에 업로드 합니다. (컴파일 오류 또는 Arduino와의 USB 연결 오류가 발생하면 4번 영역에 표시)
3. 시리얼 모니터 버튼 : USB로 아두이노와 연결된 상태로 아두이노가 동작하면 아두이노에서 PC로 메시지를 보낼 수 있습니다. Serial.write() 또는 Serial.print()같은 함수를 사용하여 프로그램이 작성된 경우 시리얼 모니터를 통해 메시지를 확인할 수 있습니다.
4. 메시지 영역 :아두이노 개발환경을 사용하면서 발생하는 알림 메시지나 컴파일, 업로드 결과 등이 표시되는 영역입니다.

3.3. 아두이노 라이브러리 추가

1. 당사 웹사이트(<http://www.mightyzap.com>) API 자료실에서 IRROBOT_EZController_XXXX.Zip을 내려 받습니다.
(사용하는 액추에이터가 Position control제품인지, Force Control제품군인지 확인후 그에 맞는 자료실을 선택해 주세요)
2. [스케치] - [라이브러리 포함하기] - [.ZIP 라이브러리 추가...] 선택

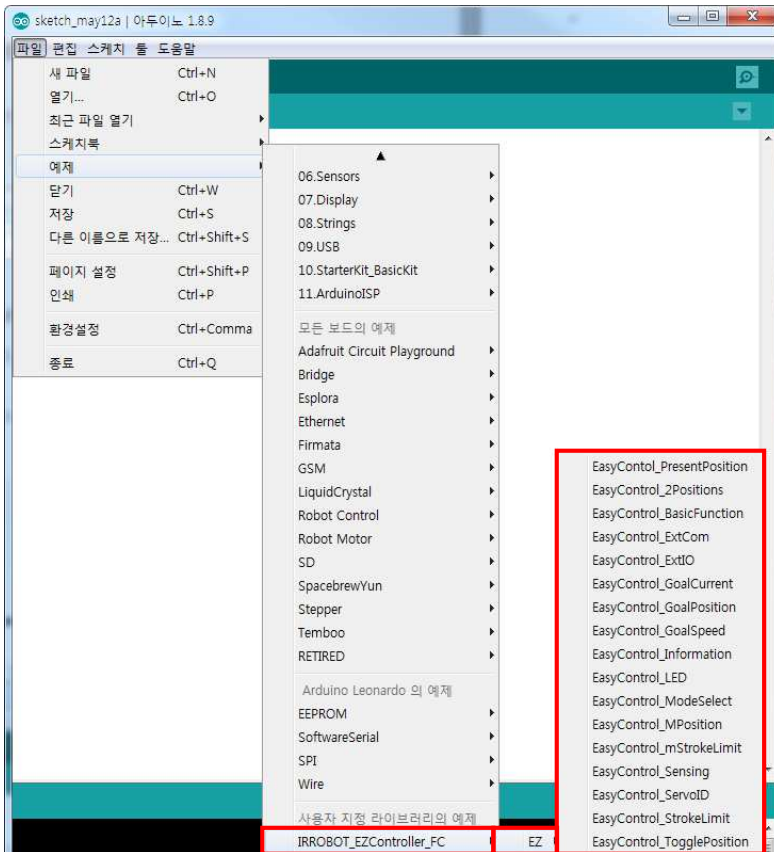


3. IRROBOT_EZController_XXXX.Zip을 선택 (파일명은 업데이트에 따라 변경될 수 있습니다.)



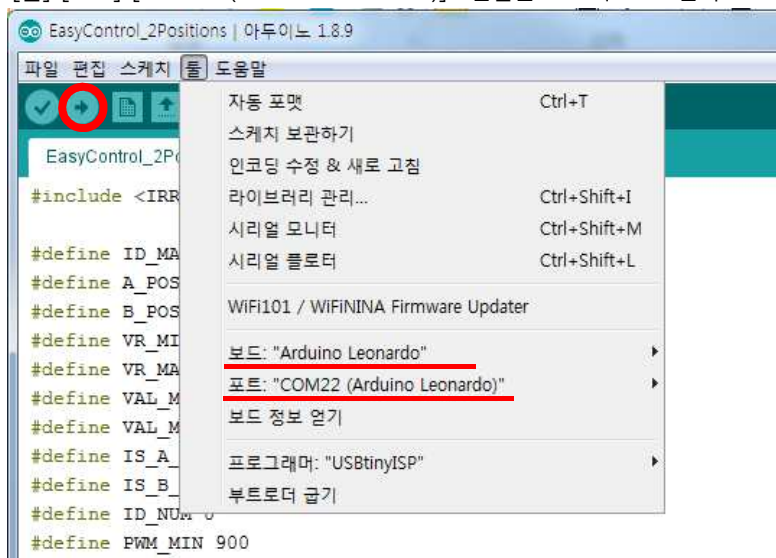
3.4. 예제 불러오기

1. 아두이노 IDE 실행
2. [파일] - [예제] - [IRROBOT_EZController_FC]-[EZ] - 사용할 예제 선택



3.5. 프로그램 업로드 하기

1. [툴]-[보드]-[Arduino Leonardo] 선택
2. [툴]-[포트]-[COMXX (Arduino Leonardo)] 연결된 보드의 포트 선택



3. 업로드 버튼을 통해 실행된 프로그램을 보드에 업로드 시켜줍니다.

4 아두이노 IDE를 통한 액추에이터 제어 예제

아두이노 IDE의 데이터 통신 프로토콜(RS-485 또는 TTL)을 통해 액추에이터를 제어합니다. 데이터 통신을 통해 액추에이터를 보다 자유롭게 제어하고 싶을 때 사용합니다.

4.1 개요

EZ Controller를 통해 mightyZAP을 제어하기 위한 아두이노 예제입니다. 본 예제는 시리얼 데이터 통신(TTL 혹은 RS-485 통신)을 통한 제어가 가능하며 PWM 통신은 지원하지 않습니다.

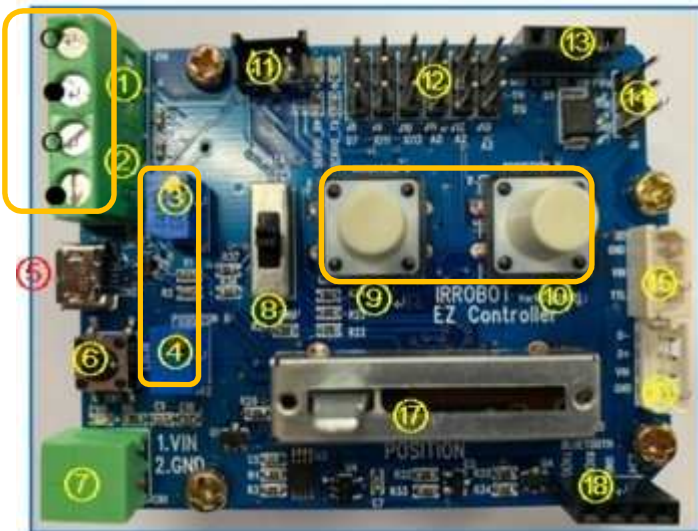
EZ Controller는 아두이노 레오나르도 기반으로 제작된 보드이며, 당사에서 제공한 아두이노 API는 아두이노 레오나르도/우노 기준입니다. 각 파라미터 기능에 대한 구체적인 설명은 마이티잡 사용자 매뉴얼을 참고하여 주십시오.

4.2. 예제 - 2 Position Control (RS-485/TTL)

두 지점을 지정할 수 있는 가변저항 2개(그림 3,4번)와 지정한 지점에 도달하도록 동작시킬 수 있는 Switch 2개(그림 9,10번)를 내장하고 있습니다. 또는 외부 물리 스위치, 외부전압신호 (그림 1번 또는 2번)로도 지정한 지점에 도달할 수 있도록 할 수 있습니다.

가변저항으로 지정시킨 2Position을 2개의 스위치를 통해 이동시키는 예제입니다.

[예제] - [IRROBOT_EZController] - [EZ]-[EZController_2Positions] 선택



[사용설명]

- 모드 스위치(8번)는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC전원(7.4V or 12V)과 GND를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터(14번)의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- A, B 지점의 위치를 조절하는 가변저항(3, 4번)을 시계(-), 반시계(+) 방향으로 돌려 동작시킬 지점을 설정합니다. 시계 방향은 수축(short stroke), 반시계 방향은 확장(long stroke) 방향입니다.
- A, B 스위치(9, 10번)를 눌러 원하는 위치로의 명령을 내립니다.
- 좌측 상단의 단자(1, 2번)를 이용하여 외부 스위치·신호를 이용하여 동작시킬 시, 5페이지를 참고하여 주십시오.

- [프로그램 설명]

```

#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define A_POSITION_VR Tester.VR_2
#define B_POSITION_VR Tester.VR_3
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define IS_A_POSITION_ON Tester.POS_A.isOFF()
#define IS_B_POSITION_ON Tester.POS_B.isOFF()
#define ID_NUM 0

#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

IRROBOT_EZController Tester(&Serial1);

void setup() {
  Tester.begin();
  Tester.Mightyzap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}

void loop() {
  unsigned char MightyZap_actID = ID_NUM;
  short A_stroke_val, B_stroke_val, position_val;
  int A_stroke_limit, B_stroke_limit;

  A_stroke_val = map(A_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  B_stroke_val = map(B_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);

  if(IS_A_POSITION_ON) position_val = A_stroke_val;
  else if(IS_B_POSITION_ON) position_val = B_stroke_val;

  Tester.Mightyzap.goalPosition(MightyZap_actID, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(15);
}

```

가변 저항의 값으로 두 지점을 설정하며 A와 B의 버튼이 눌렸을 때 해당 지점으로 움직이게 해주는 예제

PWM 값을 위한 맵핑
Short : 900 Long : 2100

Position A와 Position B의 가변 저항 값을 읽어 A_stroke_val 변수와 B_stroke_val 변수에 각각 값을 할당해 줍니다

map() 함수를 통해 가변저항의 저항 값을 액츄에이터의 위치 범위에 맞게 매핑시켜줍니다.

*가변저항 이용하지 않을 시,

A_stroke_val = 100;
B_stroke_val =
3600;

4096의 Scale을 가지며 0~4096 사이 값 기입해줍니다.

Tester.MightyZAP.goalPosition() : TTL, 485 제어
Tester.servof_CH1.writeMicroseconds() : PWM 제어

4.3. 예제 - TogglePosition

1 개의 스위치로 지정시킨 2 Position 을 반전시켜주는 예제 입니다.

[예제] - [IRROBOT_EZController] - [EZ]- [EasyControl_TogglePosition] 선택

[사용설명]

- 모드 스위치(8번)는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC전원(7.4V or 12V)과 GND를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터(14번)의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- A, B 지점의 위치를 조절하는 가변저항(3, 4번)을 시계(-), 반시계(+) 방향으로 돌려 동작시킬 지점을 설정합니다. 시계 방향 은 수축(short stroke), 반시계 방향은 확장(long stroke) 방향입니다.
- A 또는 B 스위치(그림 9 또는 10번)를 눌러 원하는 위치로의 명령을 내립니다.
- A와 B 스위치 상관없이, 동일한 스위치를 반복하여 누르면 A->B 지점으로 B->A 지점으로 액추에이터가 이동합니다.
- 좌측 상단의 단자(1, 2 번)를 이용하여 외부 스위치·신호를 이용하여 동작시킬 시, 5 페이지를 참고하여 주십시오.

[프로그램 설명]

```
#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define A_POSITION_VR Tester.VR_2
#define B_POSITION_VR Tester.VR_3
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define IS_A_POSITION_ON Tester.POS_A.isOFF()
#define IS_B_POSITION_ON Tester.POS_B.isOFF()
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

IRROBOT_EZController Tester(&Serial1);

short position_val;
bool tg_flag;

void setup() {
  Tester.begin();
  Tester.MightyZap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}

void loop() {
  unsigned char MightyZap_actID = ID_NUM;
  short A_stroke_val;
  short B_stroke_val;

  A_stroke_val = map(A_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  B_stroke_val = map(B_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);

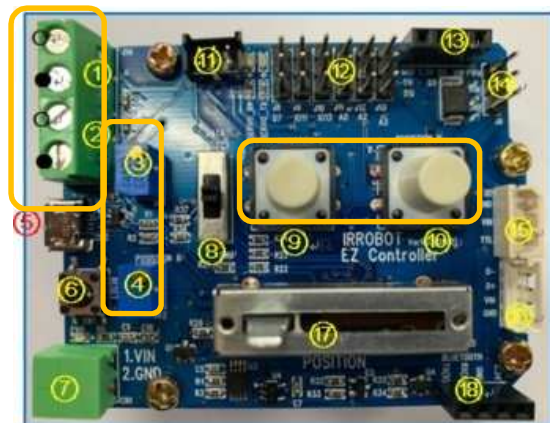
  if(IS_A_POSITION_ON || IS_B_POSITION_ON) tg_flag ^= 1;

  if(tg_flag) position_val = A_stroke_val;
  else position_val = B_stroke_val;

  Tester.MightyZap.goalPosition(MightyZap_actID, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(15);
}
```

가변저항으로 설정된 두 지점을 A나 B 버튼을 눌렀을 시에 A 지점에서 B지점으로, B지점에서 A지점으로 반전동작을 시켜주는 예제

A나 B 버튼이 눌렀을 시에 bool형으로 선언된 tg_flag 값을 반전시키고 그 값이 1일 때는 A지점을, 0일 때는 B지점으로 이동한다.



4.4. 예제 -Manual Position

Linear Potentiometer의 이동거리만큼 액추에이터의 위치를 움직여줍니다.

[예제] - [IRROBOT_EZController] - [EZ]- [EasyControl_MPosition] 선택

[사용설명]

- 모드 스위치는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC전원(7.4V or 12V)과 GND를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- 중단의 가변 Potentiometer(그림 17번) 를 조절하여 액추에이터를 동작시켜 줍니다. 매뉴얼 모드에서는 위치 가변저항 (그림 3,4번)의 설정에 영향을 받지 않고, Potentiometer가 움직이는 만큼 full stroke 구간에서 위치 제어가 됩니다.

[프로그램 설명]

```
#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define MANUAL_POSITION_VR Tester.VR_1
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

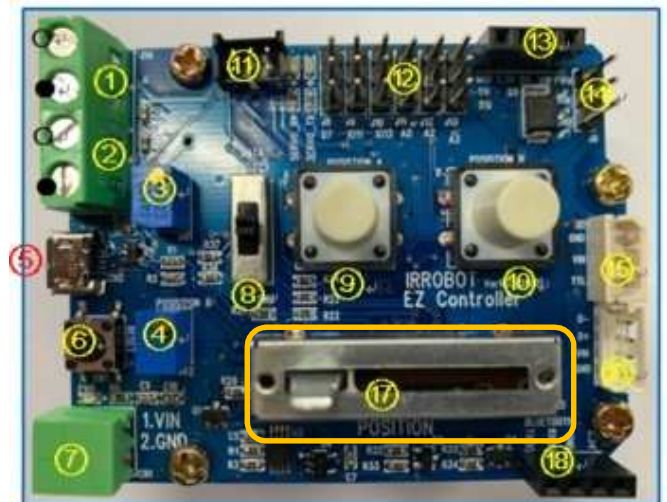
IRROBOT_EZController Tester(&Serial1);

short position_val;

void setup() {
  Tester.begin();
  Tester.MightyZap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}

void loop()
{
  unsigned char MightyZap_actID = ID_NUM;
  short Manual_position_val;
  short A_stroke_val;
  short B_stroke_val;

  Manual_position_val = map(MANUAL_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  position_val = Manual_position_val;
  Tester.MightyZap.goalPosition(MightyZap_actID, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(15);
}
```



Manual_position_val 변수로 Potentiometer의 값을 읽은 후 액추에이터 위치 동작

4.5. 예제 -Basic Function

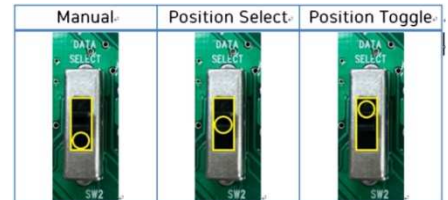
Slide 스위치를 통하여 액추에이터 제어 방식을 전환시키며 제어하는 예제입니다.

※공장 출하 프로그램

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl_BasicFunction] 선택

[사용설명]

- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC전원(7.4V or 12V)과 GND를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6 페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- 모드 슬라이드 스위치(그림 8)로 Mode를 변환시켜 액추에이터를 동작시켜 줍니다.
 - 1) Mode0 : Manual Control (스위치 최하단)
 - 2) Mode1 : 2 Position Control (스위치 중간단)
 - 3) Mode2 : Toggle Control (스위치 최상단)
- Mode1(Manual Control) 동작 시, 그림3, 4번의 가변저항 값이 17번 가변 Potentiometer의 가동범위 Limit 값으로 설정 되니 참고바랍니다.



[프로그램 설명]



```

if(short_stroke_limit>long_stroke_limit)
{
    int temp = short_stroke_limit;
    short_stroke_limit = long_stroke_limit;
    long_stroke_limit = temp;
}

if(IS_MANUAL_MODE_ON)
{
    if(Manual_position_val<short_stroke_limit) Manual_position_val = short_stroke_limit;
    else if(Manual_position_val>long_stroke_limit) Manual_position_val = long_stroke_limit;
    position_val = Manual_position_val;
}
else if(IS_2P_MODE_ON)
{
    if(IS_A_POSITION_ON) position_val = A_stroke_val;
    else if(IS_B_POSITION_ON) position_val = B_stroke_val;
}
else if(IS_TOGGLE_MODE_ON)
{
    if(IS_A_POSITION_ON || IS_B_POSITION_ON) {
        if(!Sw_status){
            if(sw_cnt++>7){
                tg_flag ^= 1;
                Sw_status = 1;
                sw_cnt = 0;
            }
        }
        else sw_cnt = 0;
    }
    else {
        sw_cnt = 0;
        if(!IS_A_POSITION_ON && !IS_B_POSITION_ON){
            if(cnt++>7){
                cnt = 0;
                Sw_status = 0;
            }
        }
    }
    if(tg_flag == 1) position_val = A_stroke_val;
    else position_val = B_stroke_val;
}

Tester.MightyZap.goalPosition(MightyZap_actID,position_val);
Tester.servo_CH1.writeMicroseconds(PWM_VAL);
delay(10);
}
  
```

```

#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define MANUAL_POSITION_VR Tester.VR_1
#define A_POSITION_VR Tester.VR_2
#define B_POSITION_VR Tester.VR_3
#define IS_MANUAL_MODE_ON Tester.MODE_0.isOFF()
#define IS_2P_MODE_ON Tester.MODE_1.isOFF()
#define IS_TOGGLE_MODE_ON Tester.MODE_2.isOFF()
#define IS_A_POSITION_ON Tester.POS_A.isOFF()
#define IS_B_POSITION_ON Tester.POS_B.isOFF()

#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)
  
```

Slide Switch의 하단, 중단, 상단이 각 순서대로 MODE_0, MODE_1, MODE_2로 정의되어 있습니다.

변수 Counting을 이용한 Debounce
Sw_status : 스위치가 눌러져 있는 상태 판별
sw_cnt : SW On시, Debouncing 후 인식
cnt : SW Off시, Off 동작 판별 후 인식

MODE_0 : Manual Position(Potentiometer)
MODE_1 : Position Select(A, B Position)
MODE_2 : Position Toggle(A,B Position)

4.6. 예제 -Extra IO(1)

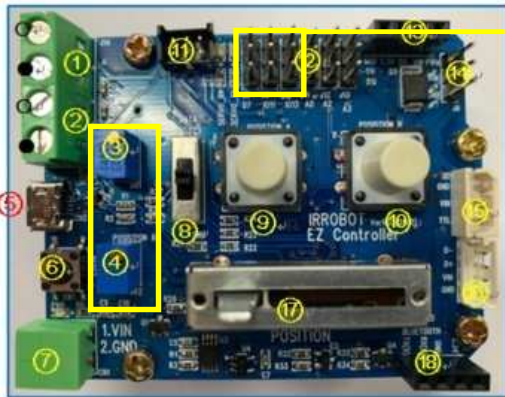
상단의 Digital IO 핀의 입력을 받아 액추에이터를 제어하는 예제 입니다.

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl_ExtIO] 선택

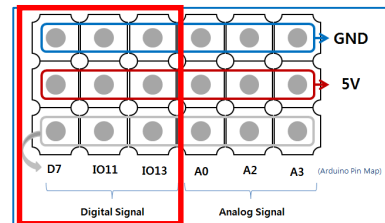
[사용설명]

- 모드 스위치는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC전원(7.4V or 12V)과 GND를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- 상단의 헤더핀(그림 12 번) 중 좌측 3 개는 Digital 단자이며, 이 중에서 좌측에서부터 첫 번째와 두 번째 핀을 사용합니다.
- 신호 단자는 3 핀 중 제일 하단의 있는 핀이며 해당 핀은 5V 의 신호 레벨을 가집니다. (아래 그림 참조)
(Active Low / 5V : High Signal / 0V : Low Signal)
- 좌측 두 개의 가변저항(그림 3,4 번)를 통해 이전방법과 동일하게 이동시킬 두 지점을 설정해줍니다.
- 첫 번째 핀에 Low 신호를 가하게 되면 설정한 A 지점으로 액추에이터가 이동하며,
두 번째 핀에 Low 신호를 가하게 되면 설정한 B 지점으로 액추에이터가 이동합니다.

[프로그램 설명]



●	GND
●	5V
●	SIG



Push Button Switch 위에 6개의 부가적인 IO 핀이 있으며 좌측 3개는 Arduino의 Digital IO, 우측 3개는 Analog IO핀 입니다.

Digital IO (7,11,13)중 7번과 11번 IO를 입력으로 선언

```
#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define A_POSITION_VR Tester.VR_2
#define B_POSITION_VR Tester.VR_3
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

IRROBOT_EZController Tester(&Serial1);

short position_val;

void setup() {
  pinMode(7, INPUT);
  pinMode(11, INPUT);
  Tester.begin();
  Tester.MightyZap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}

void loop() {
  unsigned char MightyZap_actID = ID_NUM;
  short A_stroke_val, B_stroke_val;

  A_stroke_val = map(A_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  B_stroke_val = map(B_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);

  if(digitalRead(7) == HIGH) position_val = A_stroke_val;
  else if(digitalRead(11) == HIGH) position_val = B_stroke_val;
  else position_val = position_val;
  Tester.MightyZap.goalPosition(MightyZap_actID, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(15);
}
```

7번핀에 5V 신호가 인가되면 액추에이터는 설정한 지점A로 이동하며 11번핀에 5V 신호가 인가되면 액추에이터는 설정한 지점B로 이동한다.

4.7. 예제 -Extra IO(2)

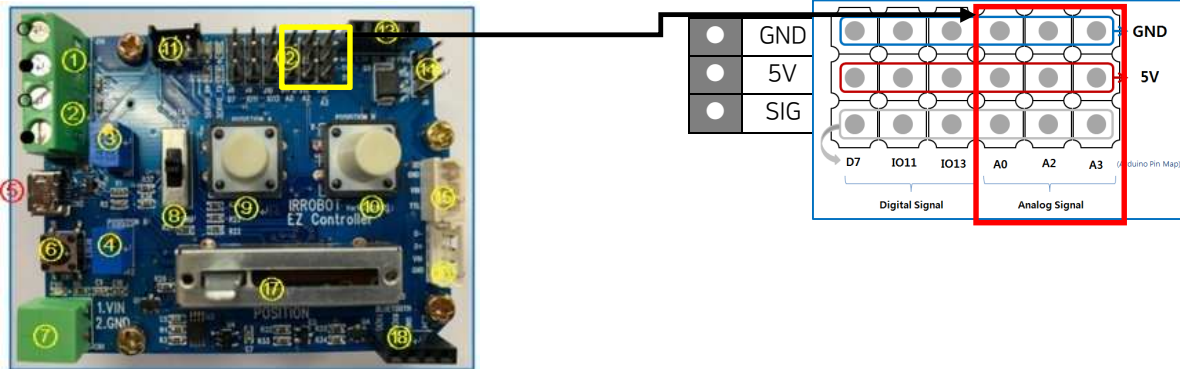
상단 Analog IO핀에 외부 센서의 입력을 받아 액추에이터를 제어하는 예제 입니다.

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl_Sensing] 선택

[사용설명]

- 모드 스위치는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7 번)에 사용하고 하는 액추에이터에 맞는 VCC 전원(7.4V or 12V)과 GND 를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- 상단의 헤더 핀(그림12번) 중 우측 3개는 Analog 단자이며, 본 우측에서 3개의 핀을 사용합니다. (아래 그림 참조)
- 해당 핀에 Analog 값을 읽을 센서를 용도에 알맞게 연결시켜줍니다.
- 센서의 값에 따라 액추에이터가 이동하게 됩니다.
ex) CDS => 밝을수록 전진, 어두울수록 후진 동작

[프로그램 설명]



```
#include <IRROBOT_EZController.h>

IRROBOT_EZController Controller(&Serial1);

#define ID_MAX 11
#define MANUAL_POSITION_VR Controller.VR_1
#define A_POSITION_VR Controller.VR_2
#define B_POSITION_VR Controller.VR_3
#define EXT_ANALOG_VR Controller.VR_4 //VR4 : A0 //VR5 : A2 //VR6 : A3
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

short position_val;

void setup(){
  Controller.begin();
  Controller.MightyZap.begin(32);
  Controller.setStep(ID_MAX, 0, 1023);
}

void loop() {
  unsigned char MightyZap_actID = ID_NUM;
  short Ext_analog_val;

  Ext_analog_val = map(EXT_ANALOG_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  position_val = Ext_analog_val;
  Controller.MightyZap.goalPosition(ID_NUM, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(15);
}
```

라이브러리 내부적으로 상단의 아날로그핀 A0,A2,A3가 각각 VR_4,VR_5,VR_6으로 정의되어 있으며 사용하고자 하는 핀을 정의되어있는 이름으로 선언 또는 매칭되어 있는 아날로그 핀을 직접 선언하여 사용합니다.

4.8. 예제 -External Communication

우측 하단의 Bluetooth 포트를 통해 외부 통신을 하며 액추에이터를 제어하는 예제 입니다..

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl_ExtCom] 선택

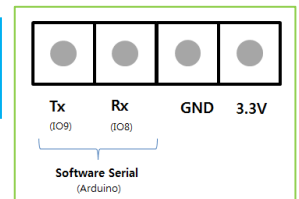
[사용설명]

- 모드 스위치는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7번)에 사용하고 하는 액추에이터에 맞는 VCC 전원(7.4V or 12V)과 GND 를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- 우측 하단 외부 통신을 위한 핀(그림 18번)이 있으며 해당 핀에 연결하고자 하는 장비를 핀 배열에 맞게 연결시켜줍니다.
- 해당 예제에선 A버튼을 누르면 ASCII값으로 'A'가 송신되고 B버튼을 누르면 ASCII값 'B'가 송신됩니다.
보드가 ASCII값으로 'A'를 수신하면 이전과 동일하게 설정해두었던 A 지점으로 액추에이터가 이동하며,
마찬가지로 ASCII값 'B'를 수신하면 설정해두었던 B지점으로 액추에이터가 이동합니다.

[프로그램 설명]



Bluetooth 기준으로 통신 전압 레벨은 3.3V를 사용합니다.



```
void Sw_Func(void){
  if(IS_A_POSITION_ON){
    userSerial.write('A');
    Serial.println("'A' Send");
    delay(500);
  }
  else if(IS_B_POSITION_ON){
    userSerial.write('B');
    Serial.println("'B' Send");
    delay(500);
  }
}

void ExtComData_Listen(void){
  if(userSerial.available()>0){
    RxChar = userSerial.read();
  }
}

bool ExtComData_Func(){
  if(RxChar == 'A'){
    position_val = A_stroke_val;
    Serial.println("'A' Recieved");
  }
  else if(RxChar == 'B'){
    position_val = B_stroke_val;
    Serial.println("'B' Recieved");
  }
  Controller.MightyZap.goalPosition(ID_NUM, position_val);
  Controller.servo_CH1.writeMicroseconds(PWM_VAL);
}
```

```
#include <IRROBOT_EZController.h>

IRROBOT EZController Controller(iSerial1);
SoftwareSerial userSerial(8,9);

#define ID_MAX 11
#define MANUAL_POSITION_VR Controller.VR_1
#define A_POSITION_VR Controller.VR_2
#define B_POSITION_VR Controller.VR_3
#define EXT_ANALOG_VR Controller.VR_4
#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define IS_A_POSITION_ON Controller.POS_A.isOFF()
#define IS_B_POSITION_ON Controller.POS_B.isOFF()
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

short position_val;
char RxChar;
short A_stroke_val, B_stroke_val;

void setup() {
  Controller.begin();
  Controller.MightyZap.begin(32);
  Controller.setStep(ID_MAX, 0, 1023);

  Serial.begin(9600);
  userSerial.begin(9600);
}

void loop() {
  unsigned char MightyZap_actID = ID_NUM;

  A_stroke_val = map(A_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  B_stroke_val = map(B_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  Sw_Func();
  ExtComData_Listen();
  ExtComData_Func();
}
```

아두이노 상으로 IO8,IO9 핀을 각각 RX,TX로 선언시켜줍니다.

ExtComData_Listen() 함수를 통하여 통신 데이터를 한 바이트씩 수신해줍니다.

Serial.begin()함수로 시리얼 모니터의 baud rate 설정

Sw_func()함수와 ExtComData_Func()함수에서 송수신된 데이터를 시리얼 모니터에 출력하여 확인합니다.

Sw_Func()함수에선 A와 B 각각 해당하는 ASCII코드 값을 송신하며 ExtComData_Func()함수에선 수신된 ASCII 코드값에 해당하는 Position으로 액추에이터를 이동시켜 줍니다.

4.9. 예제 - Mode Selection

좌측 Slide Switch(그림 8번)의 값에 따라 동작할 수 있는 모드를 설정할 수 있는 예제 입니다.

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl_ModeSelect] 선택

[사용설명]

- [Basic Function]과 동일하게 슬라이드 스위치 위치에 따라 동작이 다르며 Mode별로 동작을 지정할 수 있습니다.

[프로그램 설명]



각 모드에 대한 Index 값이 순차적으로 정의되어 있습니다.

Slide Switch가 하단일 때는 1, 중단일 때는 2, 상단일 때는 3으로 구분 지어 줍니다.

Tester.ModeSelect(int mode1, int mode2, int mode3, int sw);
 mode1 : sw값이 1일 때의 동작
 mode 2 : sw값이 2일 때의 동작
 mode3 : sw값이 3일 때의 동작
 sw : Slide Switch에 의해 설정된 Mode 값

```
#include <IRROBOT_EZController.h>

#define ID_MAX 11

#define ModeSW_1 Tester.MODE_0.isOFF()
#define ModeSW_2 Tester.MODE_1.isOFF()
#define ModeSW_3 Tester.MODE_2.isOFF()

#define MANUAL_MODE 1
#define POS2_MODE 2
#define TOGGLE_MODE 3
#define EXT_IO_MODE 4
#define EXT_SENSING_MODE 5
#define EXT_COM_MODE 6

IRROBOT_EZController Tester(&Serial1);

void setup() {
  Tester.begin();
  Tester.Mightyzap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}

void loop() {
  int sw_val;
  if(ModeSW_1) sw_val = 1;
  else if(ModeSW_2) sw_val = 2;
  else if(ModeSW_3) sw_val = 3;
  Tester.ModeSelect(MANUAL_MODE, POS2_MODE, TOGGLE_MODE, sw_val);
  delay(10);
}
```

4.10. 예제 -Stroke Limit

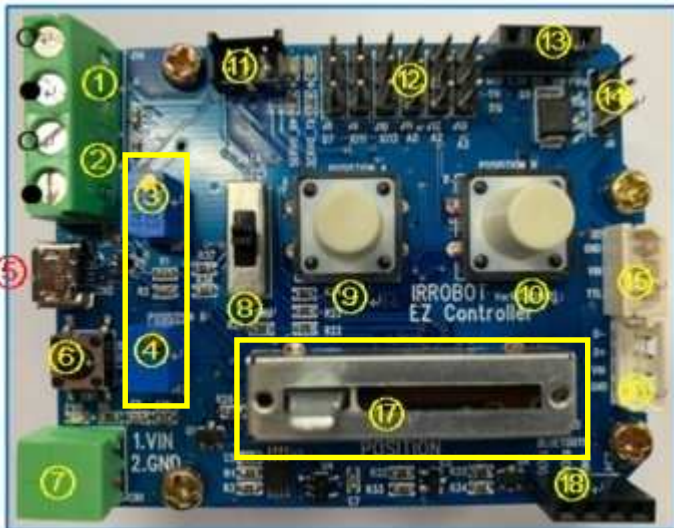
좌측 2개의 가변저항으로 Stroke의 Max/Min Limit을 설정할 수 있는 예제 입니다.

[예제] - [IRROBOT_EZController] - [EZ]-[EasyControl _StrokeLimit] 선택

[사용설명]

- 모드 스위치는 어디에 위치시켜도 무방합니다. (기본 내장 프로그램에서만 모드 스위치 동작)
- 전원 입력 단자(7 번)에 사용하고 하는 액추에이터에 맞는 VCC 전원(7.4V or 12V)과 GND 를 방향에 맞게 연결시켜 줍니다.
- 선택한 통신에 알맞은 커넥터에 방향을 유의하여 삽입해줍니다. (그림 14번 PWM / 15번 TTL / 16번 RS-485)
(PWM 커넥터의 경우 6페이지 핀맵을 참고하여 극성에 유의하여 삽입하여 주십시오)
- Stroke의 Max와 Min 지점의 위치를 조절하는 가변저항(그림 3, 4번)을 시계(-), 반시계(+) 방향으로 돌려 동작시킬 지점을 설정합니다. 시계 방향은 수축(short stroke), 반시계 방향은 확장(long stroke) 방향입니다.
- 중단의 가변 Potentiometer(그림 17번) 를 조절하여 액추에이터를 동작시켜 줍니다.
해당 프로그램에선 위치 가변저항 (그림 3,4번)의 설정에 영향을 받으며, Potentiometer를 움직여도 가변저항으로 설정한 구간에서만 위치 제어가 됩니다.

[프로그램 설명]



```
#include <IRROBOT_EZController.h>

#define ID_MAX 11
#define MANUAL_POSITION_VR Tester.VR_1
#define A_POSITION_VR Tester.VR_2
#define B_POSITION_VR Tester.VR_3
#define IS_A_POSITION_ON Tester.POS_A.isOFF()
#define IS_B_POSITION_ON Tester.POS_B.isOFF()
#define SW_A Tester.POS_A
#define SW_B Tester.POS_B

#define VR_MIN 0
#define VR_MAX 1023
#define VAL_MIN 0
#define VAL_MAX 4095
#define ID_NUM 0
#define PWM_MIN 900
#define PWM_MAX 2100
#define PWM_VAL map(position_val, VAL_MIN, VAL_MAX, PWM_MIN, PWM_MAX)

IRROBOT_EZController Tester(&Serial1);

short position_val;
bool tg_flag, Sw_status = 1;
int sw_cnt = 0;

void setup() {
  Tester.begin();
  Tester.MightyZap.begin(32);
  Tester.setStep(ID_MAX, 0, 1023);
}
```

```
void loop()
{
  unsigned char MightyZap_actID = ID_NUM;
  short Manual_position_val, A_stroke_val, B_stroke_val;
  int A_stroke_limit, B_stroke_limit, stroke_limit_dir;
  int short_stroke_limit, long_stroke_limit;

  Manual_position_val = map(MANUAL_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  short_stroke_limit = map(A_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);
  long_stroke_limit = map(B_POSITION_VR.read(), VR_MIN, VR_MAX, VAL_MIN, VAL_MAX);

  if(short_stroke_limit > long_stroke_limit)
  {
    int temp = short_stroke_limit;
    short_stroke_limit = long_stroke_limit;
    long_stroke_limit = temp;
  }

  if(Manual_position_val < short_stroke_limit) Manual_position_val = short_stroke_limit;
  else if(Manual_position_val > long_stroke_limit) Manual_position_val = long_stroke_limit;
  position_val = Manual_position_val;

  Tester.MightyZap.goalPosition(MightyZap_actID, position_val);
  Tester.servo_CH1.writeMicroseconds(PWM_VAL);
  delay(10);
}
```

상단 하단 구분 없이 큰 값이면 Max Limit,
작은 값이면 Min Limit 설정

가변 Potentiometer의 값이 설정한 범위를 벗어나지
못하도록 값 판별